

mrr_project_bio

2025-11-10

Classification of Cancer outcome using Genetic and Clinical data

Introduction

This project investigates the effect of genetic and clinical variables on the survival outcome of breast cancer patients. The data contains records of more than 1000 breast cancer patients from several research institutions. Clinical data contains patient-related and tumor-related information. Additionally mRNA gene expression data is available for each patient. The gene expression data has been processed to include only the top 5000 most variable genes on the transformed scale ($\log_2(\text{counts} + 1)$).

We consider that the main outcome variable of interest is **vital_status**, which is defined in clinical data.

Load data

```
load("mrr_bio.Rdata")
```

Genes data

```
dim(GeneX)
```

```
## [1] 1231 5000
```

```
str(GeneX)
```

```
## num [1:1231, 1:5000] 0 0 10.92 3.17 5 ...
## - attr(*, "dimnames")=List of 2
## ..$ : chr [1:1231] "EW-A2FS-01A-11R-A17B" "OL-A6VR-01A-32R-A33J" "E9-A226-01A-21R-A157" "A8-A08H-01A-21R-A00Z"
## ..$ : chr [1:5000] "CLEC3A" "SCGB2A2" "CPB1" "GSTM1" ...
```

```
colnames(GeneX)[1:10]
```

```
## [1] "CLEC3A" "SCGB2A2" "CPB1" "GSTM1" "TFF1" "SCGB1D2"
## [7] "KCNJ3" "MUCL1" "LINC00993" "ANKRD30A"
```

```
rownames(GeneX)[1:10]
```

```
## [1] "EW-A2FS-01A-11R-A17B" "OL-A6VR-01A-32R-A33J" "E9-A226-01A-21R-A157"
## [4] "A8-A08H-01A-21R-A00Z" "D8-A27H-01A-11R-A16F" "D8-A3Z6-01A-11R-A239"
## [7] "B6-A1KN-01A-11R-A13Q" "BH-A0DL-01A-11R-A115" "A8-A09X-01A-11R-A00Z"
## [10] "BH-A2L8-01A-11R-A18M"
```

GeneX is a table with 1231 rows and 5000 columns:

- The rows correspond to the patients with each row name being the patient identifier such as EW-A2FS-01A-11R-A17B.
- The columns correspond to the genes with each column name being a gene such as CLEC3A, which are the 5000 most variable genes from one person to another.
- Each cell corresponds to the expression value (a float) of a certain gene (column) for a certain patient (row) -> continuous variables

Clinical data

```
load("mrr_bio.Rdata")
dim(clinical_data)
```

```
## Loading required namespace: S4Vectors
```

```
## NULL
```

```
head(clinical_data[, 1:5])
```

```
## DataFrame with 6 rows and 5 columns
```

```
##           initial_weight tissue_type laterality
##           <numeric> <character> <character>
## EW-A2FS-01A-11R-A17B      110      Tumor      Left
## OL-A6VR-01A-32R-A33J      380      Tumor      Right
## E9-A226-01A-21R-A157      510      Tumor      Left
## A8-A08H-01A-21R-A00Z      250      Tumor      Left
## D8-A27H-01A-11R-A16F      170      Tumor      Right
## D8-A3Z6-01A-11R-A239       60      Tumor      Left
##           tissue_or_organ_of_origin age_at_diagnosis
##           <character> <integer>
## EW-A2FS-01A-11R-A17B      Breast, NOS      15302
## OL-A6VR-01A-32R-A33J      Breast, NOS      17702
## E9-A226-01A-21R-A157      Breast, NOS      16511
## A8-A08H-01A-21R-A00Z      Breast, NOS      24137
## D8-A27H-01A-11R-A16F      Breast, NOS      26588
## D8-A3Z6-01A-11R-A239      Breast, NOS      20629
```

```
colnames(clinical_data)
```

```
## [1] "initial_weight"      "tissue_type"
## [3] "laterality"          "tissue_or_organ_of_origin"
## [5] "age_at_diagnosis"    "primary_diagnosis"
## [7] "prior_treatment"     "diagnosis_is_primary_disease"
## [9] "ajcc_pathologic_t"   "morphology"
## [11] "classification_of_tumor" "sites_of_involvement"
## [13] "days_to_last_follow_up" "follow_ups_disease_response"
## [15] "race"                "gender"
## [17] "ethnicity"           "vital_status"
## [19] "age_at_index"        "days_to_birth"
## [21] "age_is_obfuscated"   "bcr_patient_barcode"
## [23] "primary_site"        "disease_type"
```

clinical_data is a table with 1231 rows and 24 columns:

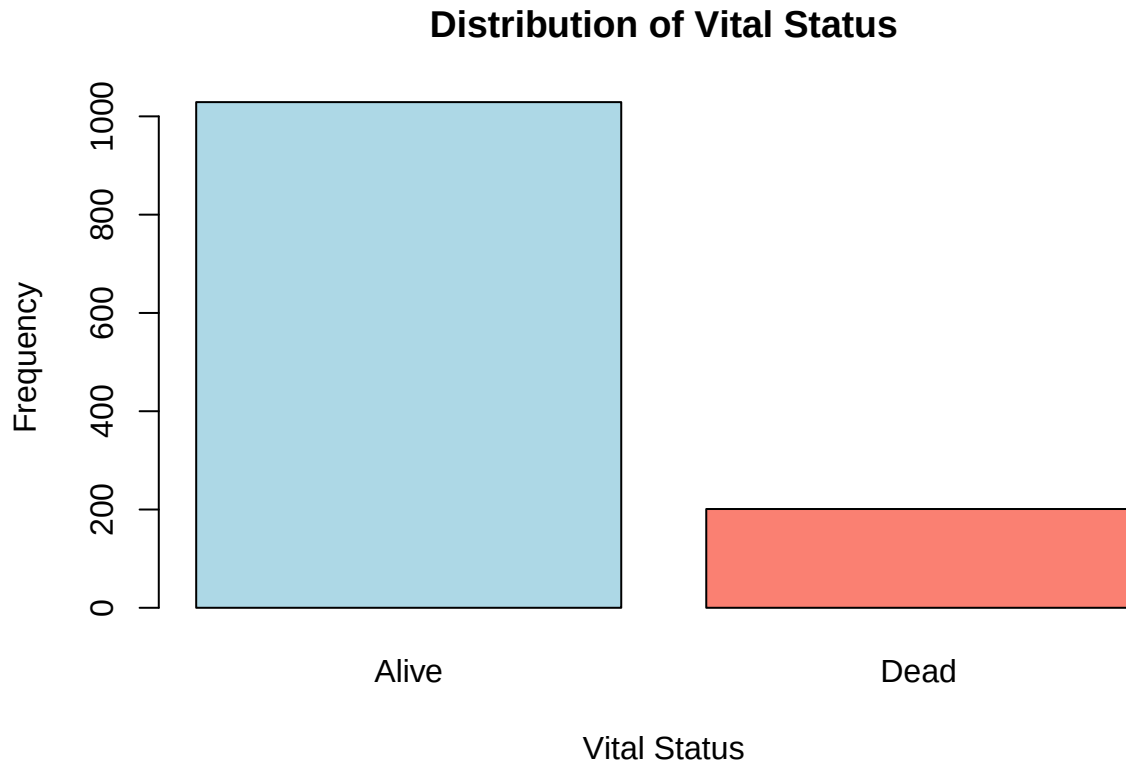
- The rows correspond to the names of the patients, similar to `GeneX`.
- The columns are the names of the clinical data, such as `initial_weight`, `sites_of_involvement`, and `vital_status`.
- Each cell corresponds to the value of the clinical data for each patient, which can be either an integer or a character string.

Outcome variable

The outcome variable is the vital status of the patient (dead or alive).

```
y <- clinical_data$vital_status
table(y)
```

```
## y
## Alive Dead
## 1029 201
barplot(table(y),
  main = "Distribution of Vital Status",
  xlab = "Vital Status",
  ylab = "Frequency",
  col = c("lightblue", "salmon"))
```



Exploratory data analysis

Genes

For the genes table, let's first plot for each gene the gaussian distribution of proportion of dead minus proportion of alive and then take the average of this graph.

```
# Calculate mean expression for each gene by vital status
for (col in colnames(GeneX)[1:5]) { # First 5 genes for demonstration
  gene_data <- GeneX[, col]
  status <- clinical_data$vital_status

  # Split data by status
  alive_expr <- gene_data[status == "Alive"]
  dead_expr <- gene_data[status == "Dead"]

  # Calculate means
  mean_alive <- mean(alive_expr, na.rm = TRUE)
  mean_dead <- mean(dead_expr, na.rm = TRUE)

  # Create density plot
```

```

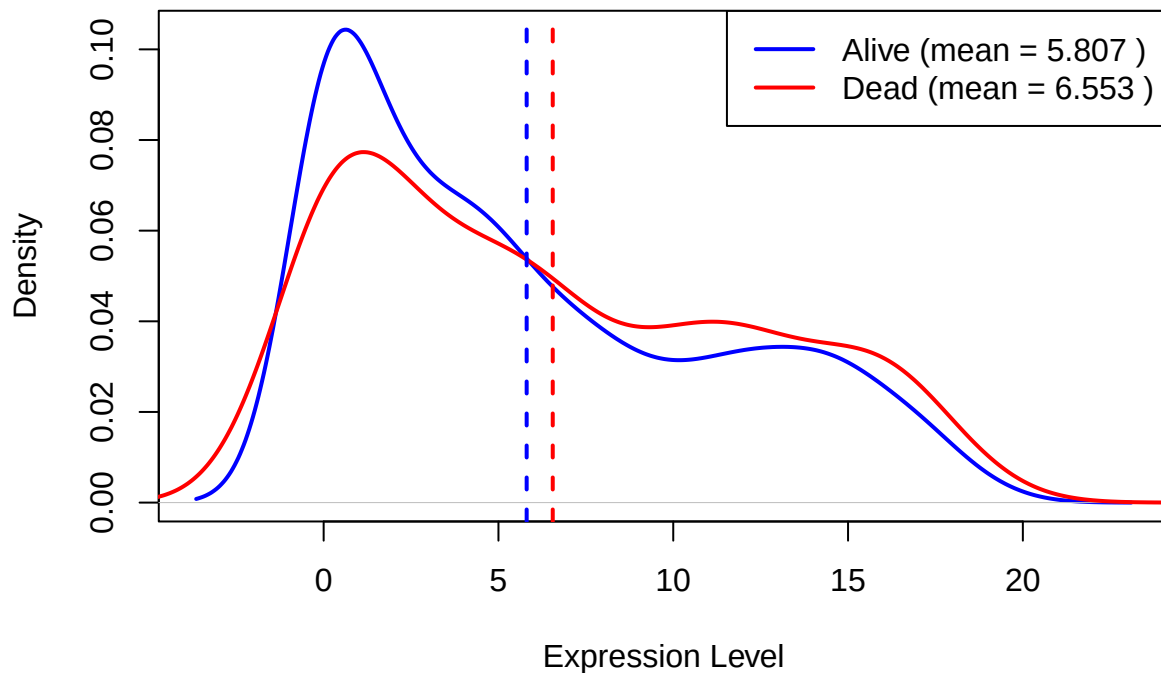
plot(density(alive_expr, na.rm = TRUE),
     col = "blue",
     lwd = 2,
     main = paste("Gene:", col),
     xlab = "Expression Level",
     ylab = "Density")
lines(density(dead_expr, na.rm = TRUE),
      col = "red",
      lwd = 2)

# Add mean lines
abline(v = mean_alive, col = "blue", lty = 2, lwd = 2)
abline(v = mean_dead, col = "red", lty = 2, lwd = 2)

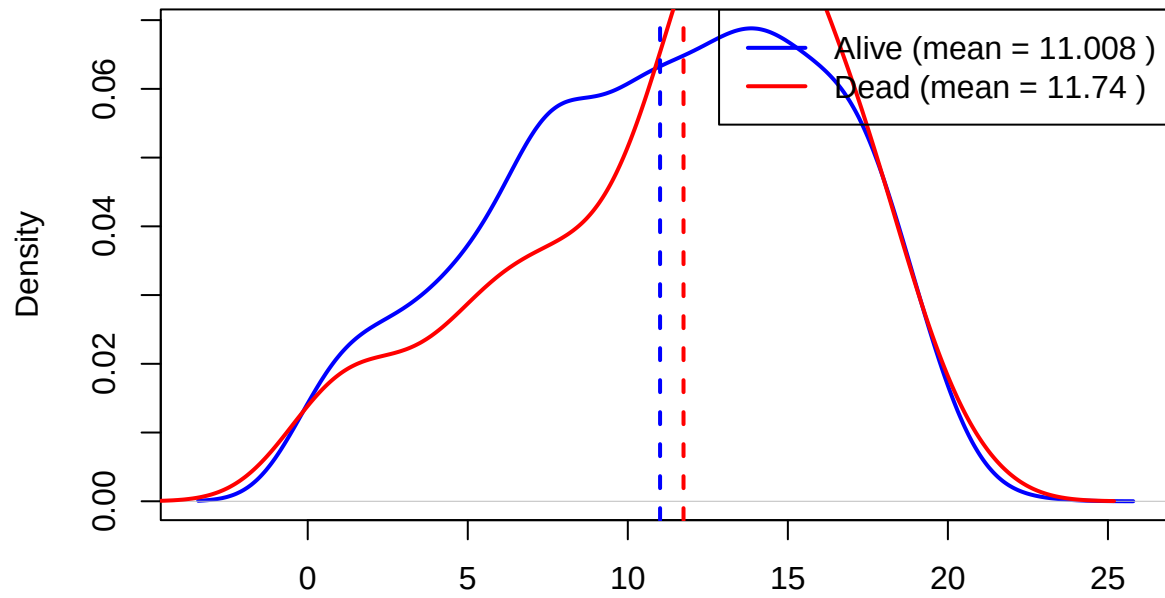
# Add legend
legend("topright",
      legend = c(paste("Alive (mean =", round(mean_alive, 3), ")"),
                 paste("Dead (mean =", round(mean_dead, 3), ")")),
      col = c("blue", "red"),
      lty = 1,
      lwd = 2)
}

```

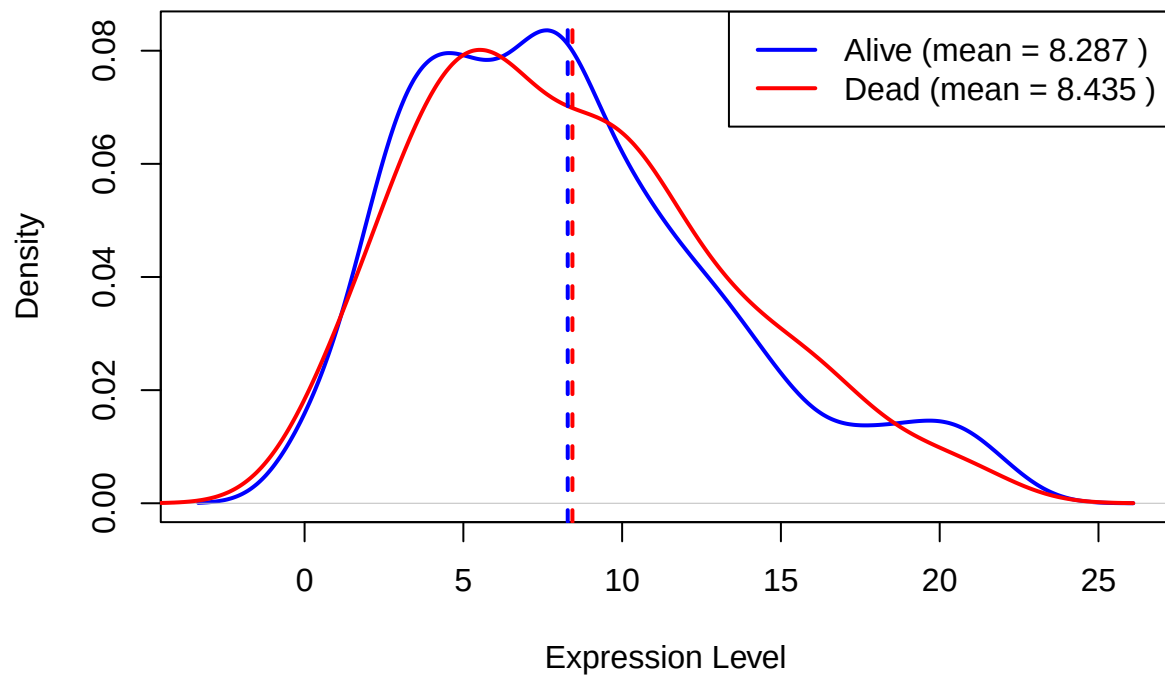
Gene: CLEC3A



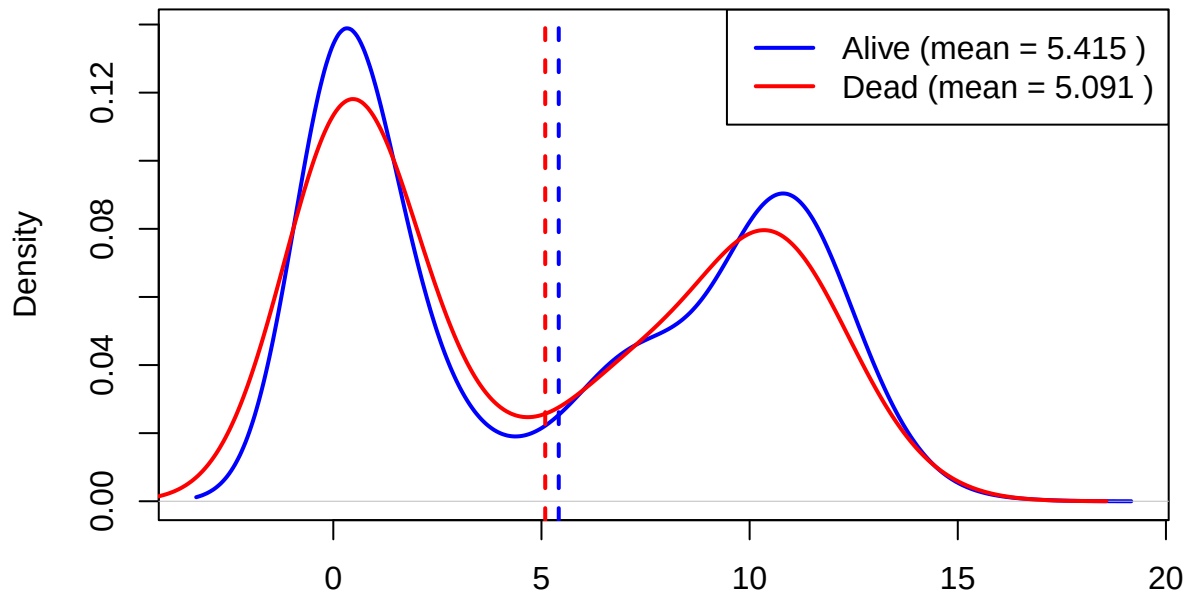
Gene: SCGB2A2



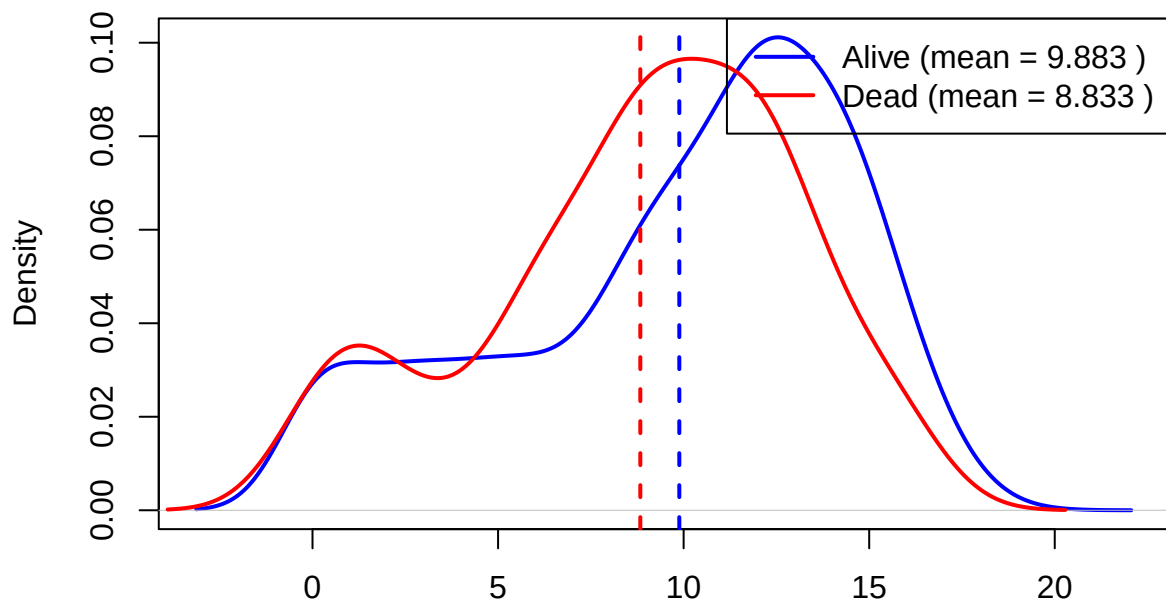
Gene: CPB1



Gene: GSTM1



Gene: TFF1



Expression Level

Thus we can display the most important genes by calculating the relative mean difference between the two groups (alive and dead) for each gene and then sorting them.

```
gene_diff <- data.frame(  
  Gene = colnames(GeneX),  
  MeanDiff = numeric(ncol(GeneX))  
)
```

```

)

for (i in seq_along(colnames(GeneX))) {
  col <- colnames(GeneX)[i]
  gene_data <- GeneX[, col]
  status <- factor(clinical_data$vital_status)

  plot_data <- data.frame(Expression = gene_data, Status = status)
  means <- aggregate(Expression ~ Status, data = plot_data, mean)

  # Calculate relative mean difference
  mean_alive <- means$Expression[means$Status == "Alive"]
  mean_dead <- means$Expression[means$Status == "Dead"]

  # Relative difference: (Dead - Alive) / Alive
  # Handle division by zero
  if (mean_alive != 0) {
    gene_diff$MeanDiff[i] <- abs((mean_dead - mean_alive) / mean_alive)
  } else {
    gene_diff$MeanDiff[i] <- NA
  }
}

# Remove genes with NA values
gene_diff <- gene_diff[!is.na(gene_diff$MeanDiff), ]

# Sort by relative mean difference
gene_diff <- gene_diff[order(gene_diff$MeanDiff, decreasing = TRUE), ]

# Display top 50 genes
top_50 <- head(gene_diff, 50)
print(top_50)

```

```

##           Gene MeanDiff
## 900      LALBA 0.8992166
## 1506     SULT1C3 0.7832339
## 2589      CSN2 0.7646311
## 4892    LINC01497 0.7279608
## 3569   AC007423.1 0.6802403
## 2308     RPS4Y1 0.5873672
## 2128     HSD3BP2 0.5872528
## 737    AC104407.1 0.5689319
## 1657   FP325317.1 0.5546457
## 2995      LHCGR 0.5456845
## 3509    LINC02237 0.5450194
## 3699     NGF-AS1 0.5247317
## 4982    LINC01612 0.5070537
## 748      CSN1S1 0.4944819
## 2205   ALDH1L1-AS2 0.4744773
## 3488      EPB42 0.4720326
## 1133   AC008459.1 0.4695645
## 192      NPY2R 0.4656152
## 3270   AL121950.1 0.4547892
## 3963   AL353693.1 0.4437907

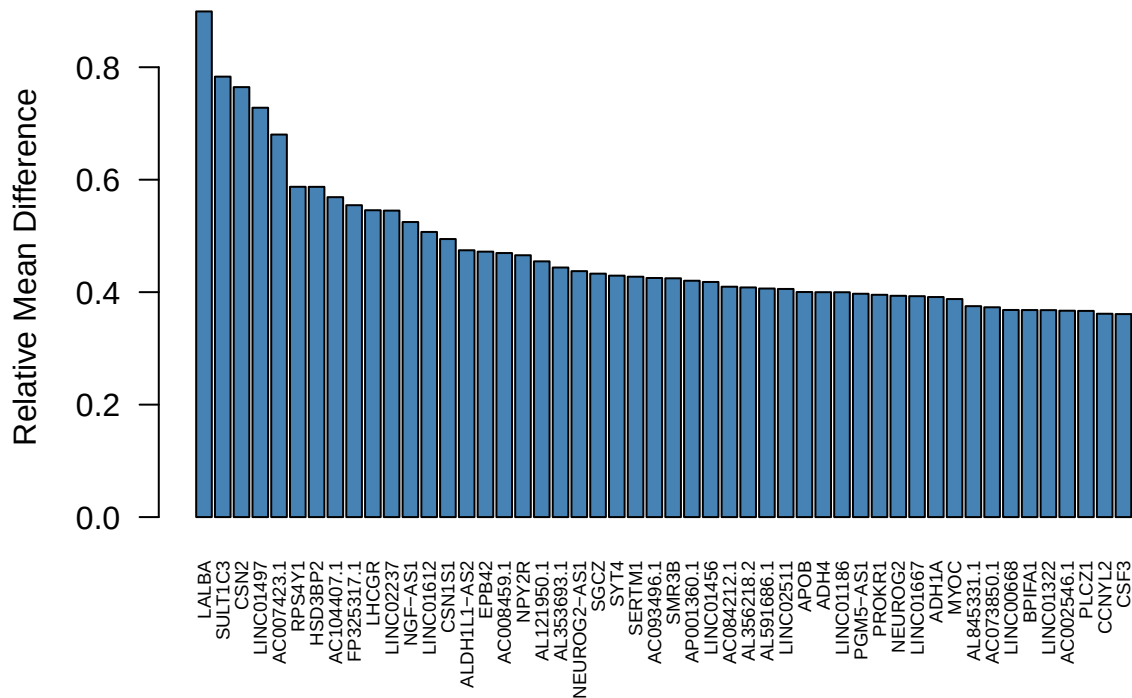
```

```
## 4992 NEUROG2-AS1 0.4373508
## 4097          SGCZ 0.4328951
## 1595          SYT4 0.4293564
## 3806          SERTM1 0.4274395
## 3696 AC093496.1 0.4251907
## 587          SMR3B 0.4246711
## 3893 AP001360.1 0.4203018
## 3438 LINC01456 0.4180961
## 4182 AC084212.1 0.4096885
## 1784 AL356218.2 0.4084072
## 2672 AL591686.1 0.4064550
## 2057 LINC02511 0.4056917
## 743          APOB 0.4002554
## 2201          ADH4 0.3999677
## 3164 LINC01186 0.3998710
## 3053 PGM5-AS1 0.3971454
## 2269 PROKR1 0.3952935
## 974          NEUROG2 0.3935187
## 1003 LINC01667 0.3928182
## 2475 ADH1A 0.3913356
## 525          MYOC 0.3877588
## 2289 AL845331.1 0.3750996
## 2805 AC073850.1 0.3730908
## 1092 LINC00668 0.3683282
## 3341 BPIFA1 0.3681901
## 4672 LINC01322 0.3680301
## 1430 AC002546.1 0.3669965
## 3273 PLCZ1 0.3666600
## 4006 CCNYL2 0.3616090
## 1772 CSF3 0.3610313
```

```
# Optional: visualize the distribution
```

```
barplot(top_50$MeanDiff[1:50],
        names.arg = top_50$Gene[1:50],
        las = 2,
        cex.names = 0.6,
        main = "Top 50 Genes by Relative Mean Difference",
        ylab = "Relative Mean Difference",
        col = "steelblue")
```


Top 50 Genes by Relative Mean Difference



Clinical Data

First modeling attempt with three variables from clinical data : `initial_weight`, `classification_of_tumor`, `morphology` to predict `vital_status`.

```
x <- clinical_data[, c("initial_weight",
                       "classification_of_tumor",
                       "morphology")]
y <- clinical_data$vital_status
model_data <- data.frame(x, vital_status = factor(y))
model_data <- na.omit(model_data) # removes NA for now
model_data$vital_status <- droplevels(model_data$vital_status)
```

Model fitting:

```
model <- glm(vital_status ~ ., data = model_data, family = binomial)
```

Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

Prediction:

```
pred <- predict(model, type = "response")
positive_class <- levels(model_data$vital_status)[2]
pred_class <- ifelse(pred > 0.5, positive_class, levels(model_data$vital_status)[1])
pred_class <- factor(pred_class, levels = levels(model_data$vital_status))
```

Confusion matrix:

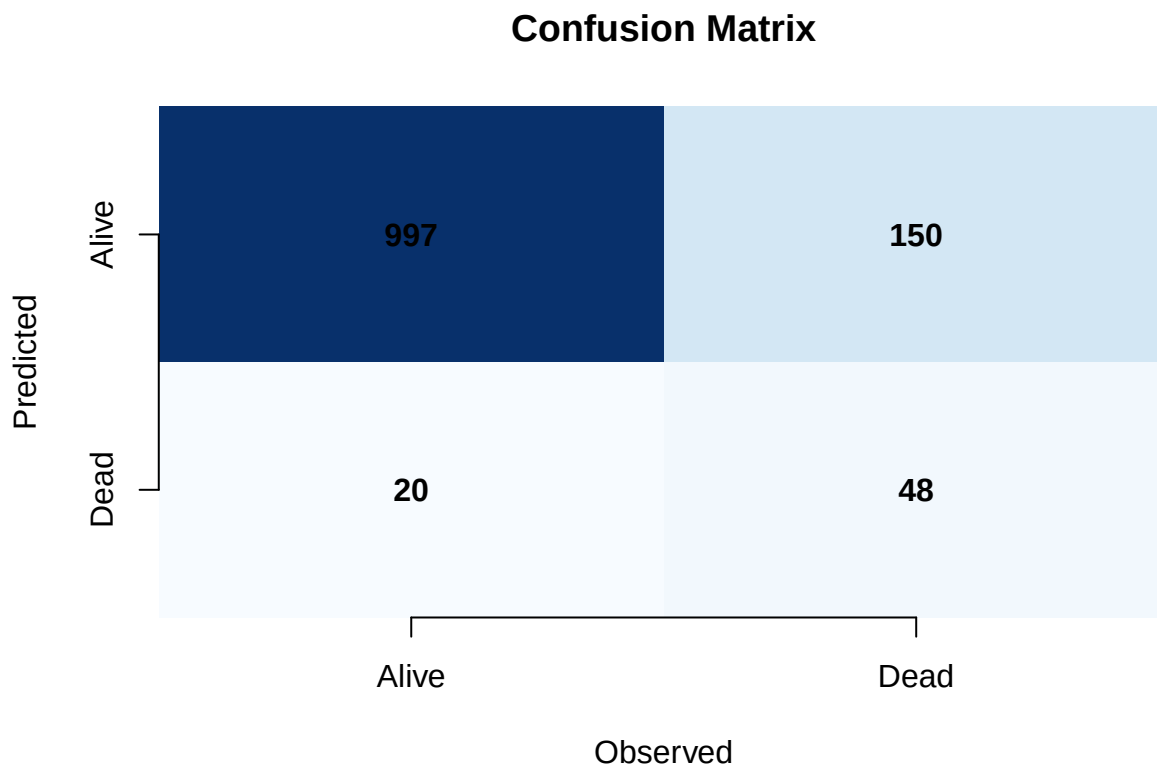
```
conf_mat <- table(Predicted = pred_class, Observed = model_data$vital_status)
conf_mat
```

```
##           Observed
```

```
## Predicted Alive Dead
##      Alive   997  150
##      Dead    20   48
```

Visualization of the confusion matrix.

```
conf_mat_matrix <- as.matrix(conf_mat)
image(t(conf_mat_matrix[nrow(conf_mat_matrix):1, ]),
      axes = FALSE,
      col = colorRampPalette(c("#f7fbff", "#6baed6", "#08306b"))(64),
      xlab = "Observed",
      ylab = "Predicted",
      main = "Confusion Matrix")
axis(1, at = seq(0, 1, length.out = ncol(conf_mat_matrix)), labels = colnames(conf_mat_matrix))
axis(2, at = seq(0, 1, length.out = nrow(conf_mat_matrix)), labels = rev(rownames(conf_mat_matrix)))
text(rep(seq(0, 1, length.out = ncol(conf_mat_matrix)), each = nrow(conf_mat_matrix)),
      rep(seq(0, 1, length.out = nrow(conf_mat_matrix)), times = ncol(conf_mat_matrix)),
      labels = as.vector(conf_mat_matrix[rev(seq_len(nrow(conf_mat_matrix))), ]),
      col = "black", font = 2)
```



Data analysis

Genes

Ce bloc pré-filtre les gènes par test univarié (top N) puis ajuste une unique régression logistique (glm) avec ces gènes simultanément. Il affiche le résumé, l'AUC et la matrice de confusion.

```
# Paramètre : nombre de gènes à garder après pré-filtrage
N <- 50

# Outcome
y <- clinical_data$vital_status
```

```

if (!is.factor(y)) y <- factor(y)
stopifnot(nrow(GeneX) == length(y))

# Pré-filtrage univarié (t-test p-values)
pv <- apply(GeneX, 2, function(g) {
  g1 <- g[y == levels(y)[1]]
  g2 <- g[y == levels(y)[2]]
  if (length(na.omit(g1)) < 2 || length(na.omit(g2)) < 2) return(1)
  res <- try(t.test(g1, g2)$p.value, silent = TRUE)
  if (inherits(res, "try-error")) return(1)
  res
})
pv[is.na(pv)] <- 1

# Top N gènes
top_genes <- names(sort(pv, decreasing = FALSE))[1:min(N, length(pv))]

# Construire jeu de données et ajuster glm (logistic)
model_df <- data.frame(vital_status = y, as.data.frame(GeneX[, top_genes, drop = FALSE]))
model_df <- na.omit(model_df)
model_df$vital_status <- droplevels(model_df$vital_status)

glm_fit <- glm(vital_status ~ ., data = model_df, family = binomial)

# Résumé
cat('\n--- Résumé du modèle (GLM) ---\n')

##
## --- Résumé du modèle (GLM) ---

print(summary(glm_fit))

##
## Call:
## glm(formula = vital_status ~ ., family = binomial, data = model_df)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -9.970e-01  1.583e+00  -0.630  0.528801
## LINC01235    1.918e-01  6.455e-02   2.972  0.002957 **
## APOB         7.976e-02  6.044e-02   1.320  0.186944
## SNORD104    -1.257e-01  7.232e-02  -1.738  0.082291 .
## RHBDL1      -6.305e-02  7.595e-02  -0.830  0.406429
## RPS4Y1      -2.207e-01  8.175e-02  -2.700  0.006940 **
## CST1        -2.421e-02  2.860e-02  -0.847  0.397253
## PSD2        2.585e-01  6.904e-02   3.745  0.000181 ***
## AC104211.1   1.470e-01  9.077e-02   1.620  0.105330
## SFTPb        -1.288e-01  5.978e-02  -2.155  0.031167 *
## LYVE1        -4.556e-02  8.256e-02  -0.552  0.581040
## AC079804.3   8.228e-02  6.246e-02   1.317  0.187738
## H2BC4        -1.004e-01  5.559e-02  -1.805  0.071011 .
## PCP2         7.771e-02  7.585e-02   1.025  0.305573
## IL4I1        -1.274e-01  7.754e-02  -1.643  0.100397
## DSC1         -5.642e-02  6.333e-02  -0.891  0.372967

```

```

## ADGRG6      1.048e-01  6.619e-02  1.583 0.113385
## AC105118.1 -5.600e-02  8.699e-02 -0.644 0.519704
## SPEF1       7.089e-03  7.854e-02  0.090 0.928077
## MAFA.AS1    3.013e-03  1.008e-01  0.030 0.976151
## LINC01186   5.366e-05  6.903e-02  0.001 0.999380
## LINC01497   -4.573e-02  7.600e-02 -0.602 0.547385
## KLB         7.822e-02  9.221e-02  0.848 0.396291
## KCNB1       -9.868e-02  6.599e-02 -1.495 0.134824
## PROKR1      7.827e-02  5.360e-02  1.460 0.144200
## LINC02511   3.160e-03  7.186e-02  0.044 0.964921
## ATF3        6.926e-02  7.033e-02  0.985 0.324725
## ADCYAP1R1  -1.172e-01  8.169e-02 -1.435 0.151323
## GPX3        6.246e-03  1.078e-01  0.058 0.953811
## BCHE        1.319e-01  4.815e-02  2.740 0.006150 **
## FOXJ1       -4.544e-02  3.744e-02 -1.214 0.224890
## AC021148.2 -2.065e-01  6.200e-02 -3.330 0.000868 ***
## GLP2R       -3.057e-02  7.009e-02 -0.436 0.662742
## TNNI3       -5.057e-02  4.102e-02 -1.233 0.217642
## TNFRSF18    5.513e-02  6.711e-02  0.821 0.411395
## HSPB6       2.809e-02  8.499e-02  0.330 0.741038
## NLGN1       1.050e-02  5.408e-02  0.194 0.845989
## CRHBP       7.104e-02  1.005e-01  0.707 0.479863
## LUARIS      -8.361e-02  7.477e-02 -1.118 0.263484
## LVRN        -3.825e-02  8.385e-02 -0.456 0.648257
## SLC2A4      -6.658e-02  7.939e-02 -0.839 0.401683
## EFEMP1      7.161e-02  7.068e-02  1.013 0.310963
## GLRA4      -5.504e-03  7.527e-02 -0.073 0.941708
## PLCXD3      5.128e-02  5.554e-02  0.923 0.355813
## DGAT2       -2.063e-02  7.373e-02 -0.280 0.779590
## ASPA        -7.227e-02  1.032e-01 -0.700 0.483659
## BTNL9       -8.010e-03  9.402e-02 -0.085 0.932104
## HPN.AS1     -8.072e-02  6.569e-02 -1.229 0.219193
## AC007423.1  2.044e-02  8.799e-02  0.232 0.816328
## HIF3A       -1.458e-01  6.482e-02 -2.249 0.024492 *
## ADH4        -3.415e-02  6.841e-02 -0.499 0.617706
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 1095.4  on 1229  degrees of freedom
## Residual deviance:  917.4  on 1179  degrees of freedom
## AIC: 1019.4
##
## Number of Fisher Scoring iterations: 6
pred_prob <- predict(glm_fit, type = 'response')

# Sauvegarder la liste des top genes utilisés
write.csv(data.frame(top_genes = top_genes), file = 'top_genes_used_in_glm.csv', row.names = FALSE)

## Graphe des classes (TP/TN en vert, FP/FN en orange)
# Convert probabilities to predicted class
# with 0 being below the median and 1 above

```

```

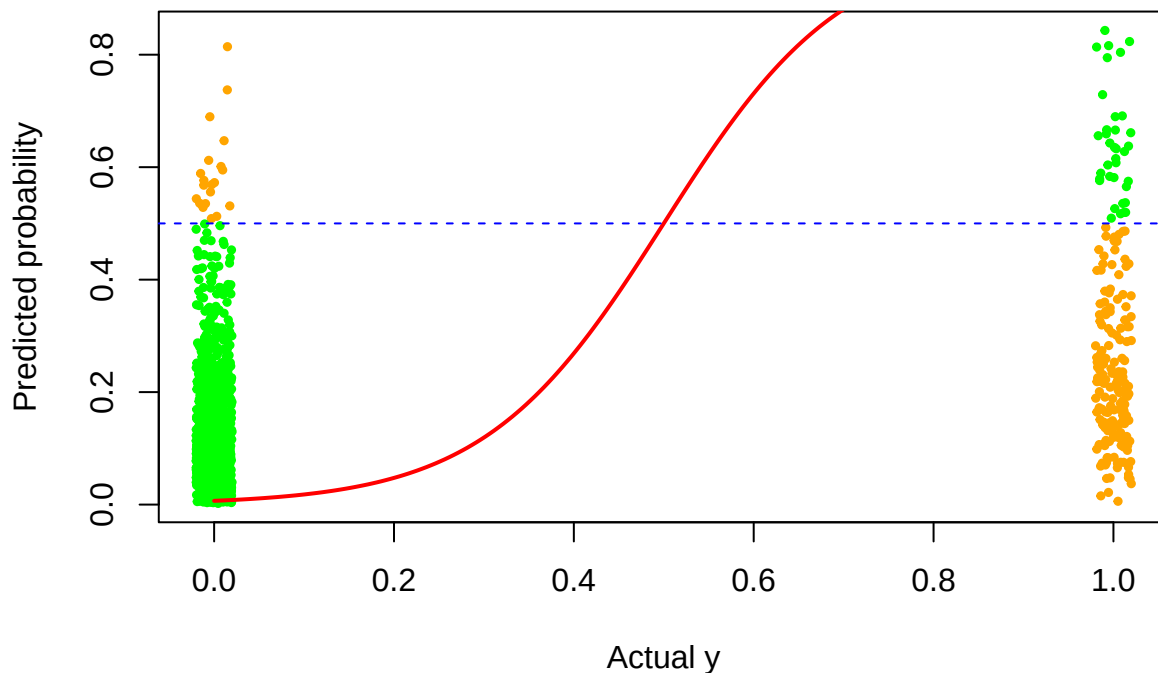
pred_resp <- pred_prob
# construire y binaire (0/1) compatible
y <- ifelse(model_df$vital_status == levels(model_df$vital_status)[2], 1, 0)
pred_class <- ifelse(pred_resp >= 0.5, 1, 0)
point_colors <- ifelse(
  pred_class == y,
  "green", # True positives / True negatives
  "orange" # False positives / False negatives
)

plot(
  y + runif(length(y), -0.02, 0.02),
  pred_resp,
  xlab = "Actual y",
  ylab = "Predicted probability",
  main = "Predicted vs Actual (Green = TP/TN, Orange = FP/FN)",
  col = point_colors,
  pch = 19,
  cex = 0.5
)

x_log <- seq(0, 1, length.out = 200)
y_log <- 1 / (1 + exp(-10 * (x_log - 0.5)))
lines(x_log, y_log, col = "red", lwd = 2)
abline(h = 0.5, col = "blue", lty = 2)

```

Predicted vs Actual (Green = TP/TN, Orange = FP/FN)



```

## Visualisation de la matrice de confusion (heatmap)
# Recalculer conf_mat pour avoir Predicted vs Observed
conf_mat_final <- table(Predicted = pred_class, Observed = model_df$vital_status)

```

```

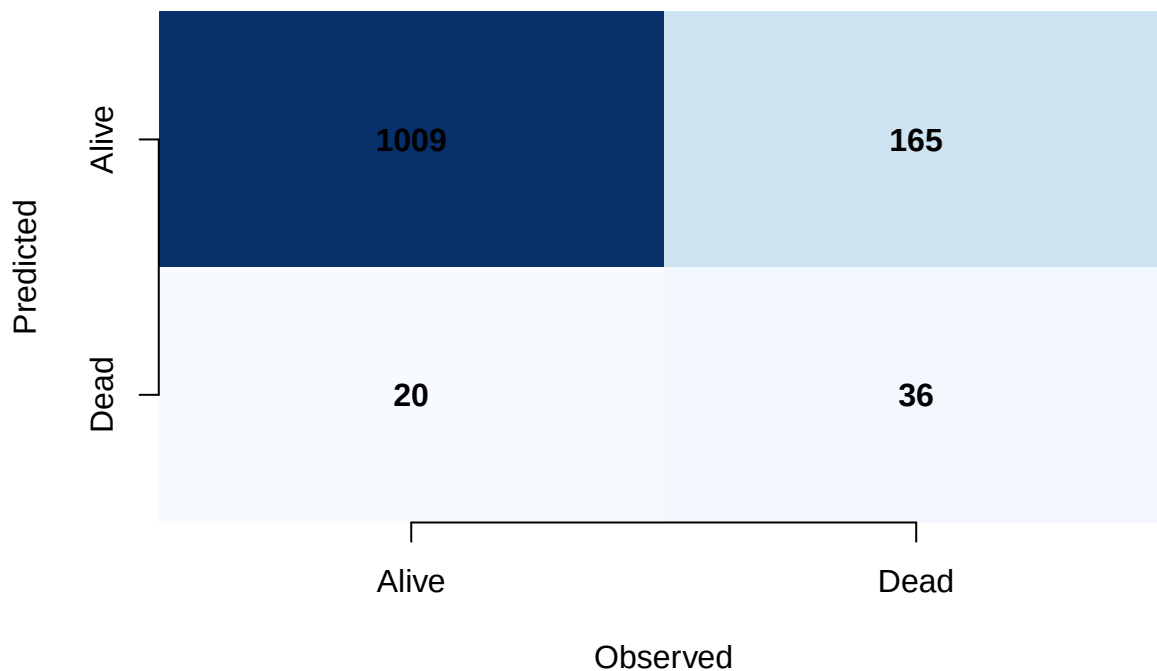
# Indicateurs simples
accuracy <- sum(diag(conf_mat_final)) / sum(conf_mat_final)
cat(sprintf("\nAccuracy: %.3f\n", accuracy))

##
## Accuracy: 0.850

# Visualisation de la matrice de confusion
conf_mat_matrix <- as.matrix(conf_mat_final)
image(t(conf_mat_matrix[nrow(conf_mat_matrix):1, ]),
      axes = FALSE,
      col = colorRampPalette(c("#f7fbff", "#6baed6", "#08306b"))(64),
      xlab = "Observed",
      ylab = "Predicted",
      main = "Confusion Matrix")
axis(1, at = seq(0, 1, length.out = ncol(conf_mat_matrix)), labels = colnames(conf_mat_matrix))
axis(2, at = seq(0, 1, length.out = nrow(conf_mat_matrix)), labels = c("Dead", "Alive"))
text(rep(seq(0, 1, length.out = ncol(conf_mat_matrix)), each = nrow(conf_mat_matrix)),
      rep(seq(0, 1, length.out = nrow(conf_mat_matrix)), times = ncol(conf_mat_matrix)),
      labels = as.vector(conf_mat_matrix[rev(seq_len(nrow(conf_mat_matrix))), ]),
      col = "black", font = 2)

```

Confusion Matrix



Clinical data

Data cleaning Here is each type corresponding to each data point as well as the frequencies of each occurrence.

After analysis we can remove redundant data like `age_at_diagnosis`, `days_to_birth` which are like the age in `age_at_index` and `bcr_patient_barcode` which is like the index of the line, and also `disease_type` and `primary_site` because they contain only one observation which is the same for all patients.

```

redundant_cols <- c("age_at_diagnosis", "days_to_birth", "bcr_patient_barcode",
                    "disease_type", "primary_site")
clinical_data <- clinical_data[, setdiff(names(clinical_data), redundant_cols)]
cat("Removed columns:", paste(redundant_cols, collapse = ", "), "\n")

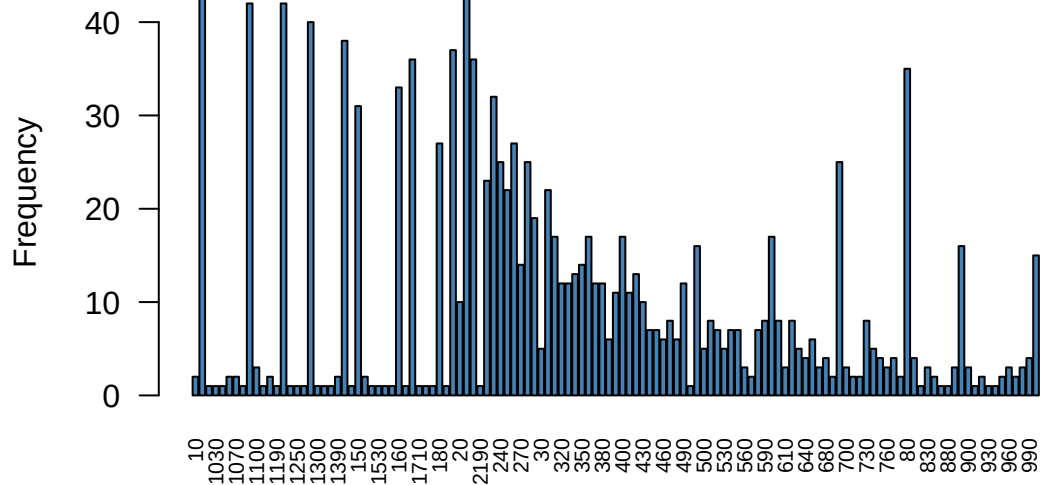
## Removed columns: age_at_diagnosis, days_to_birth, bcr_patient_barcode, disease_type, primary_site
for (col in names(clinical_data)) {
  tbl <- table(as.character(clinical_data[[col]]), useNA = "ifany")

  plot.new() # start a fresh plot for each iteration

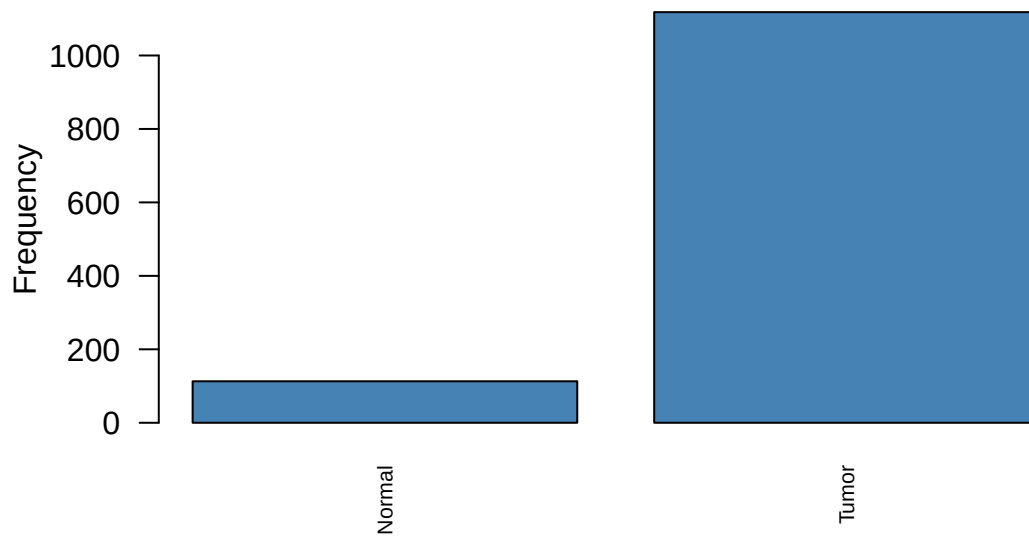
  barplot(tbl,
    main = paste("Distribution of", col),
    ylab = "Frequency",
    col = "steelblue",
    las = 2,
    cex.names = 0.7)
}

```

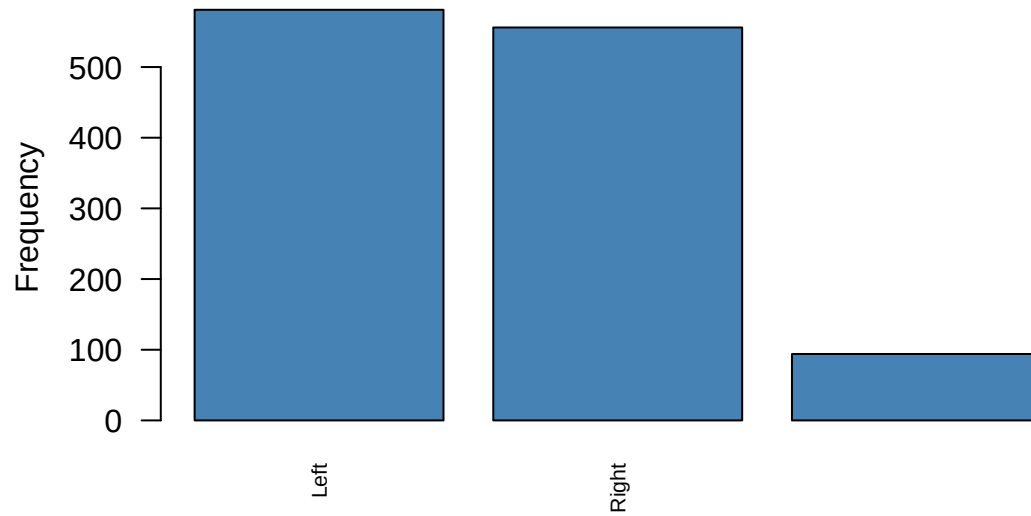
Distribution of initial_weight



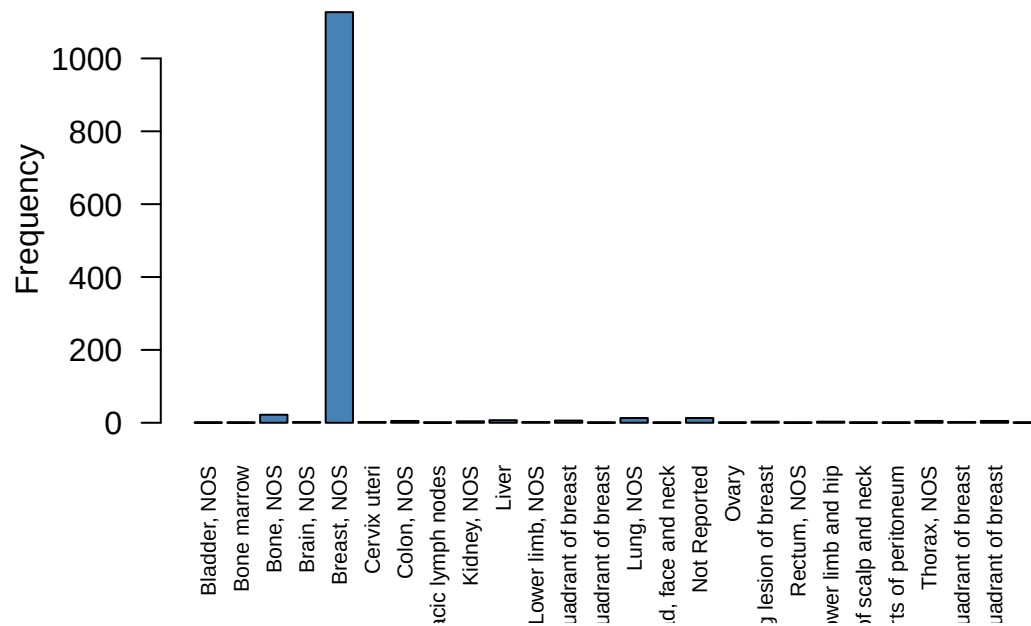
Distribution of tissue_type

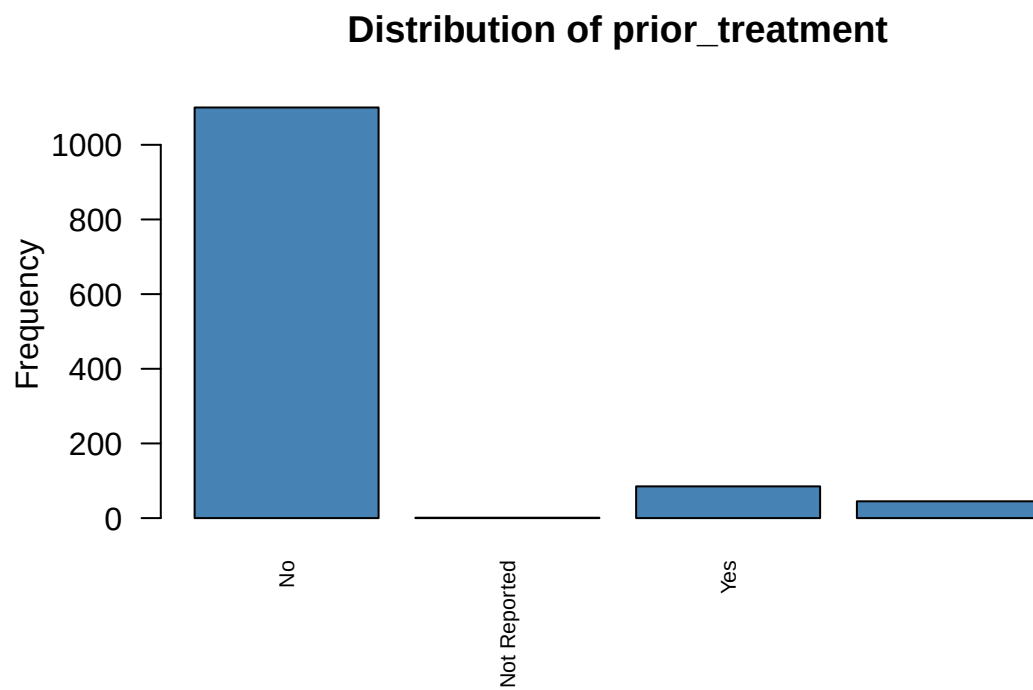
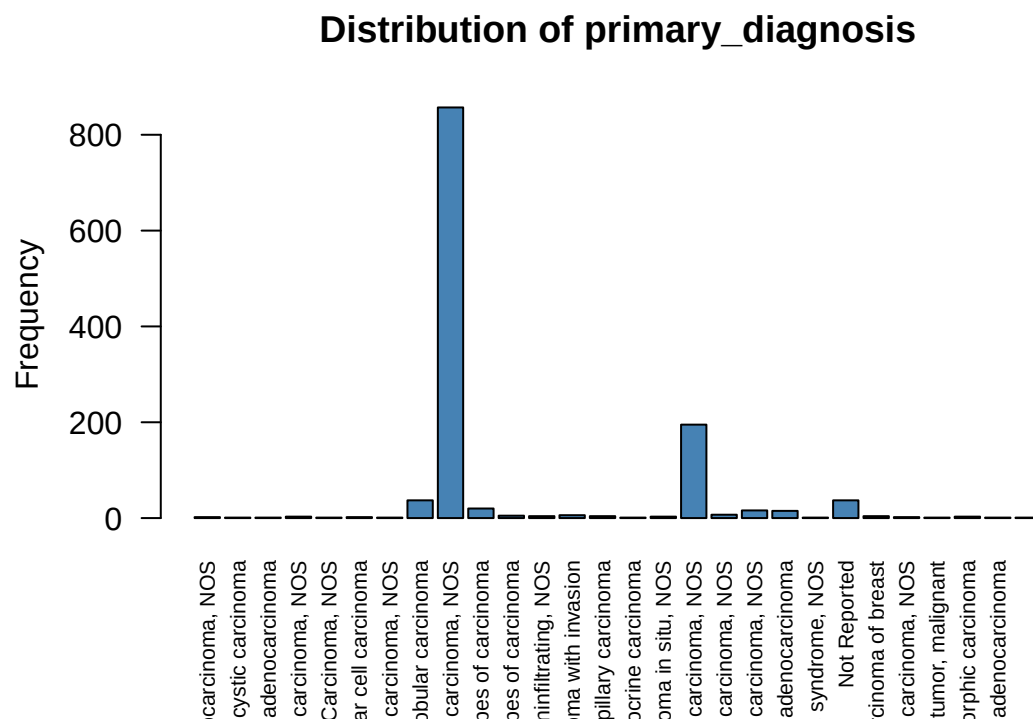


Distribution of laterality

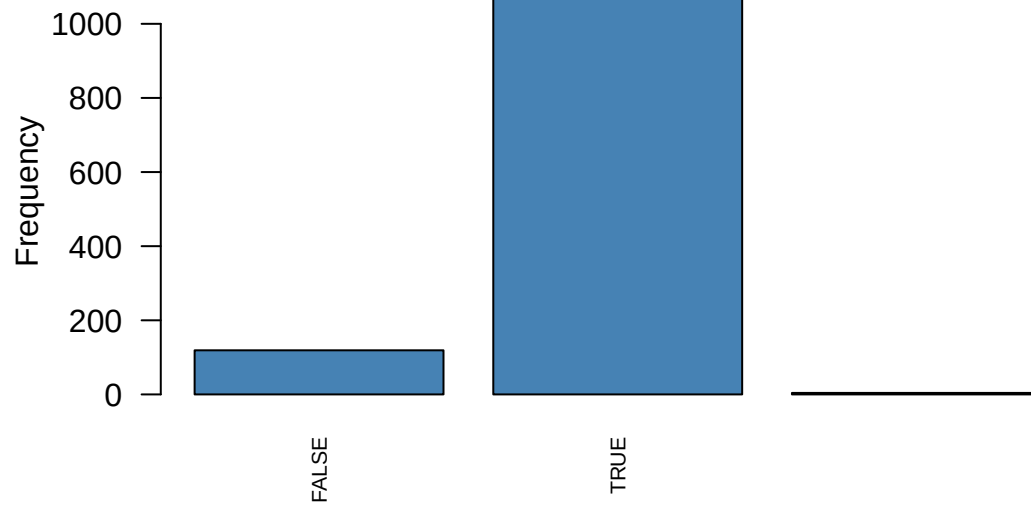


Distribution of tissue_or_organ_of_origin

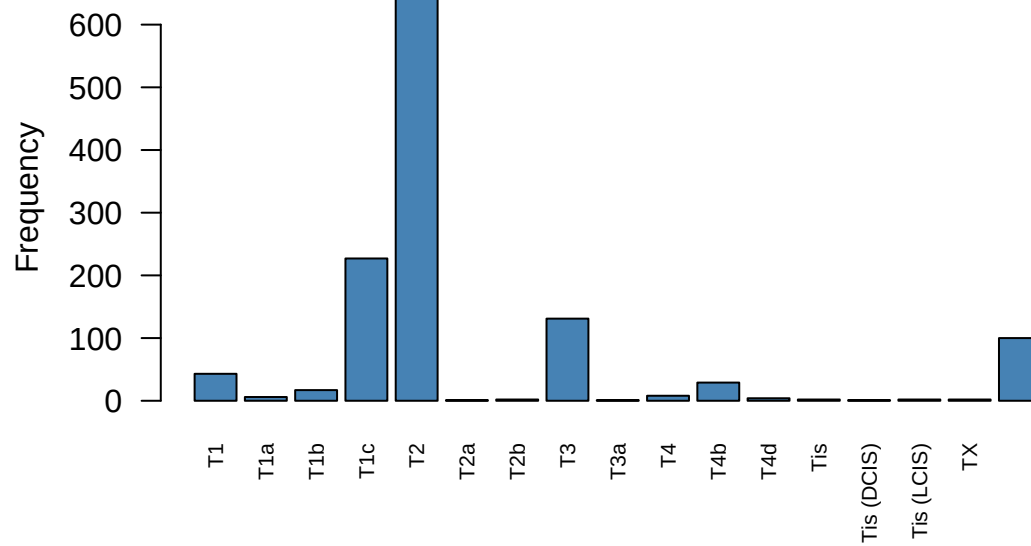




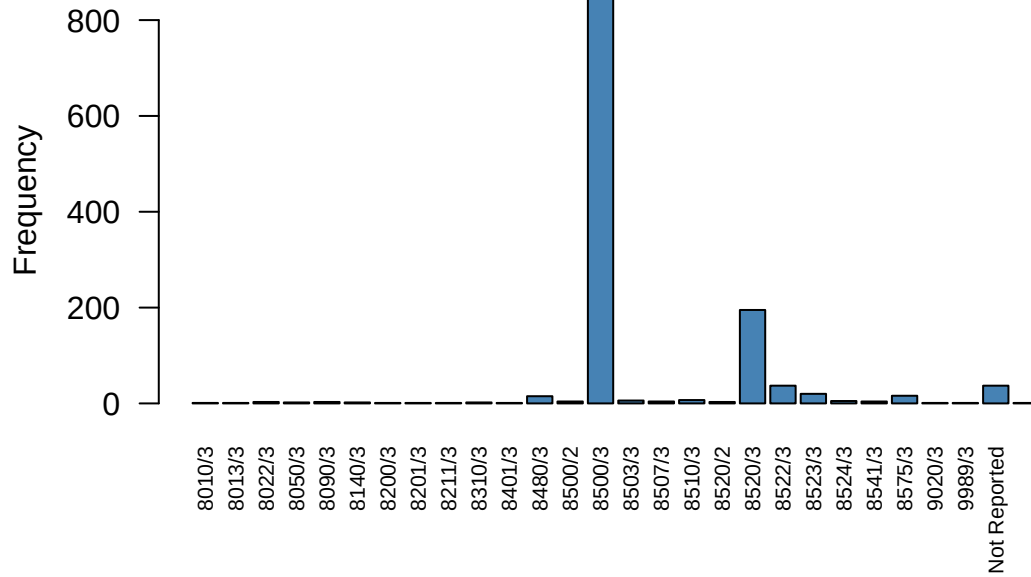
Distribution of diagnosis_is_primary_disease



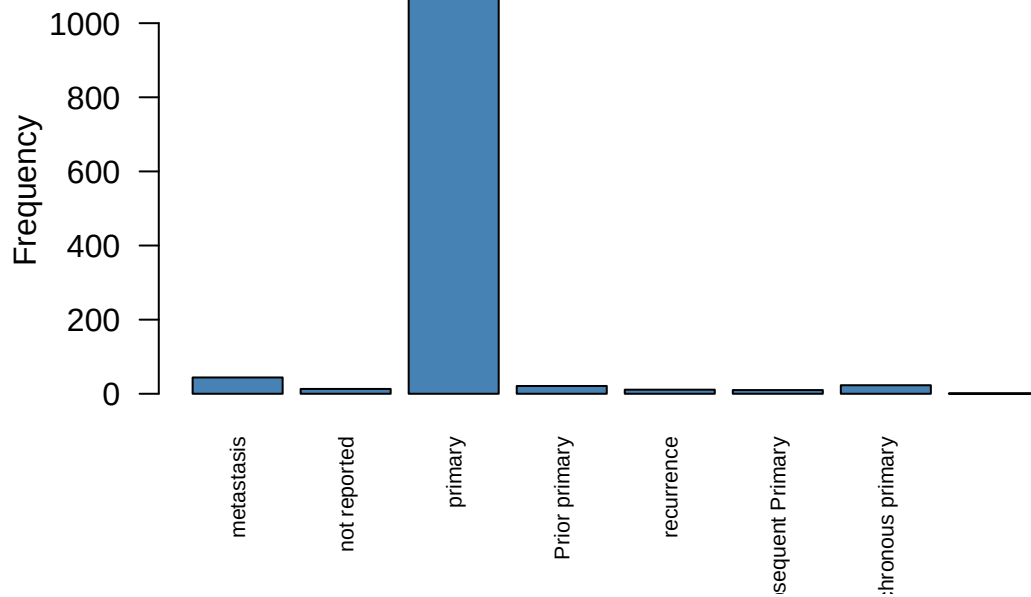
Distribution of ajcc_pathologic_t



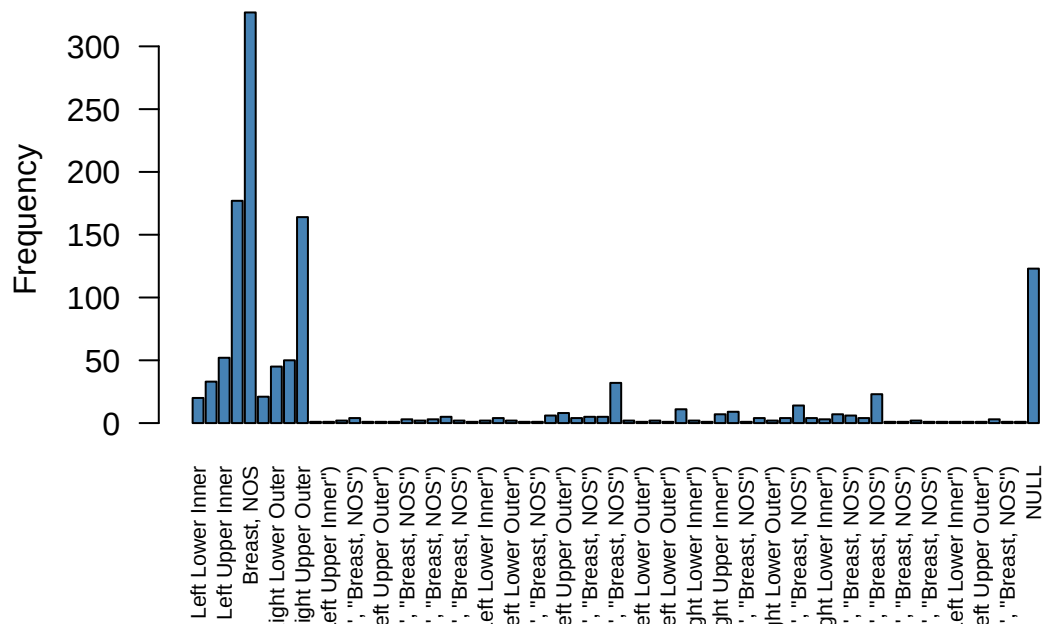
Distribution of morphology



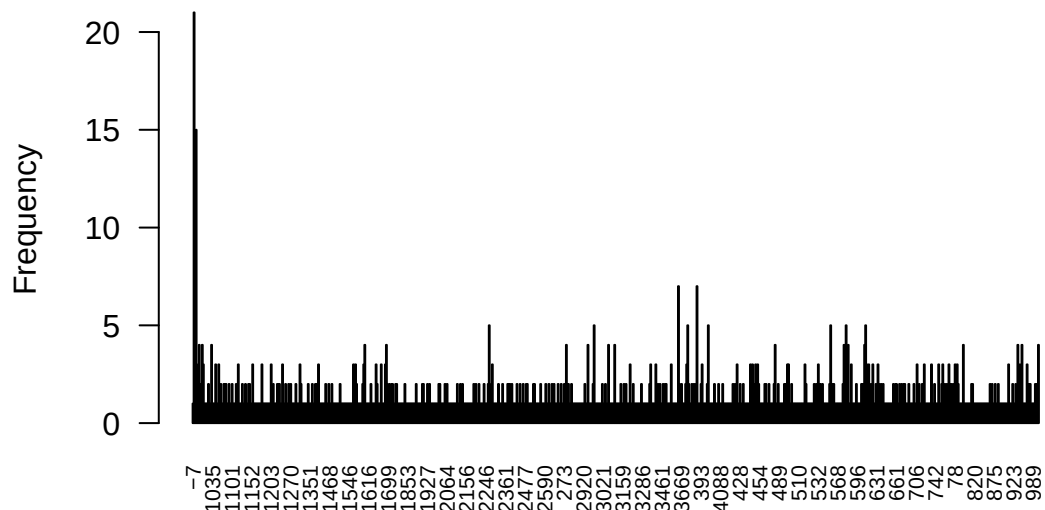
Distribution of classification_of_tumor



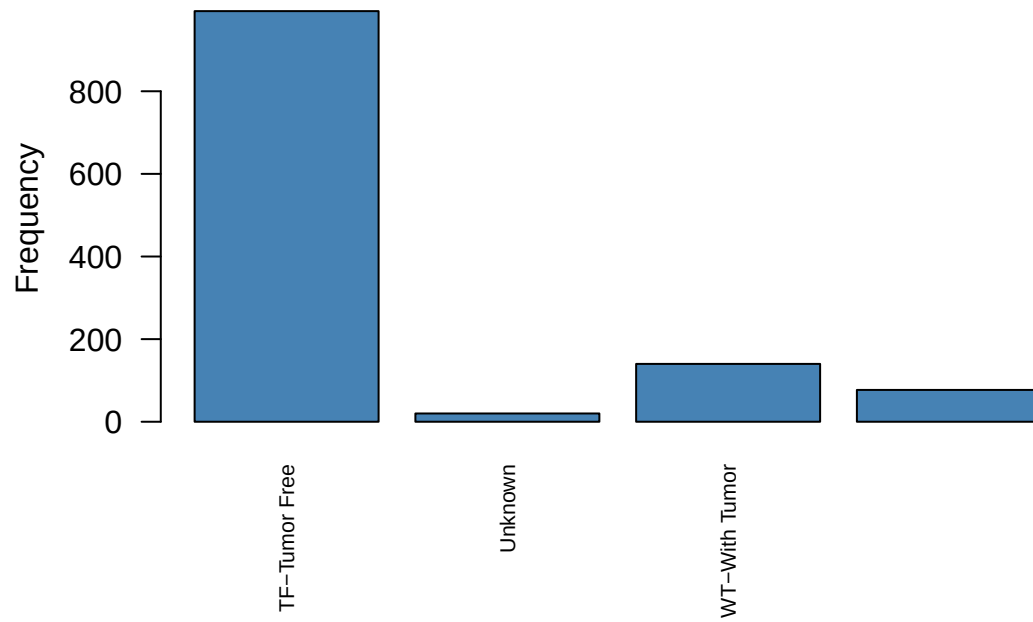
Distribution of sites_of_involvement



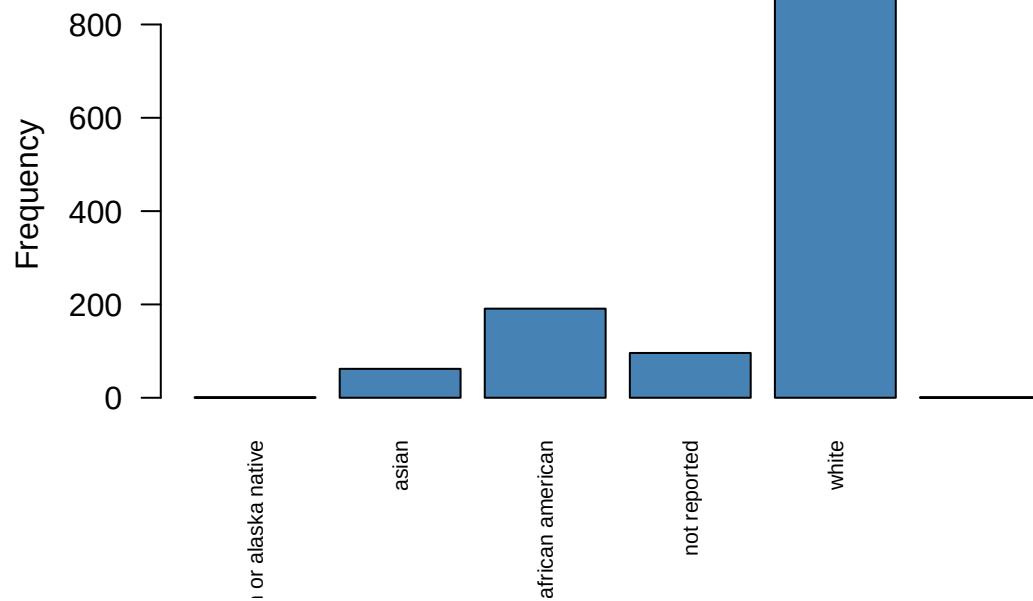
Distribution of days_to_last_follow_up



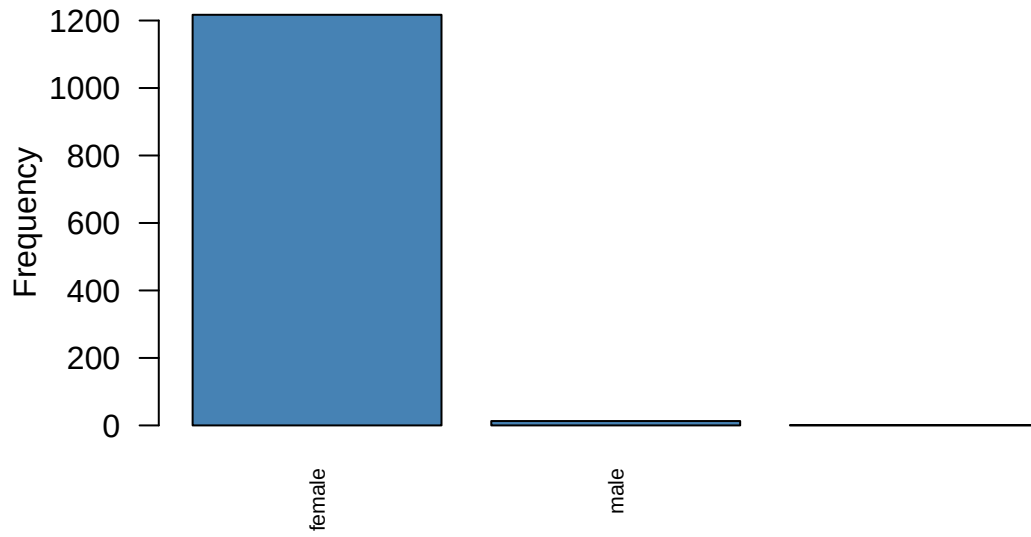
Distribution of follow_ups_disease_response



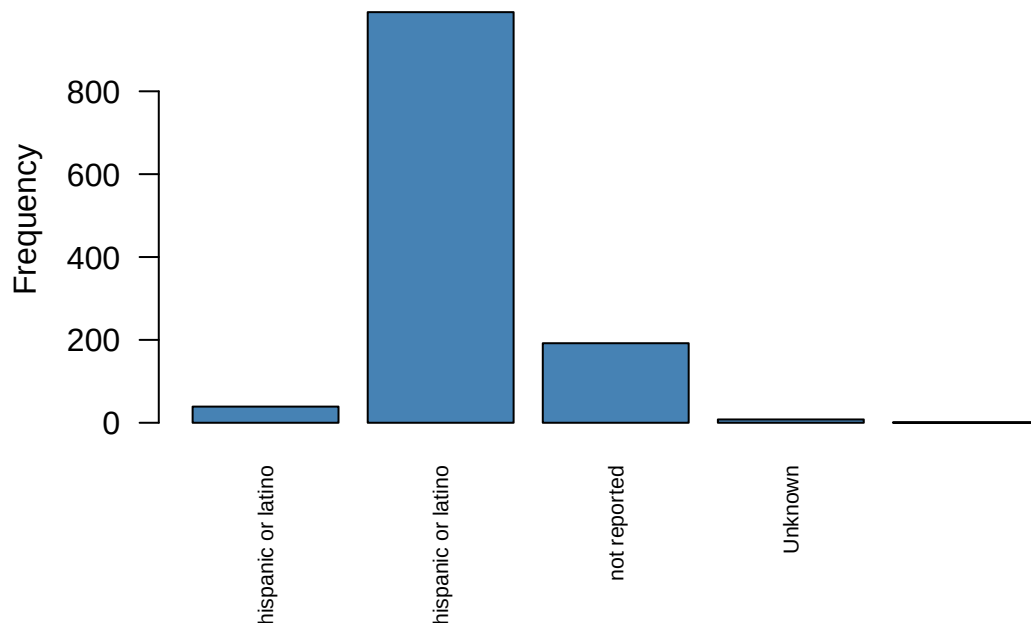
Distribution of race



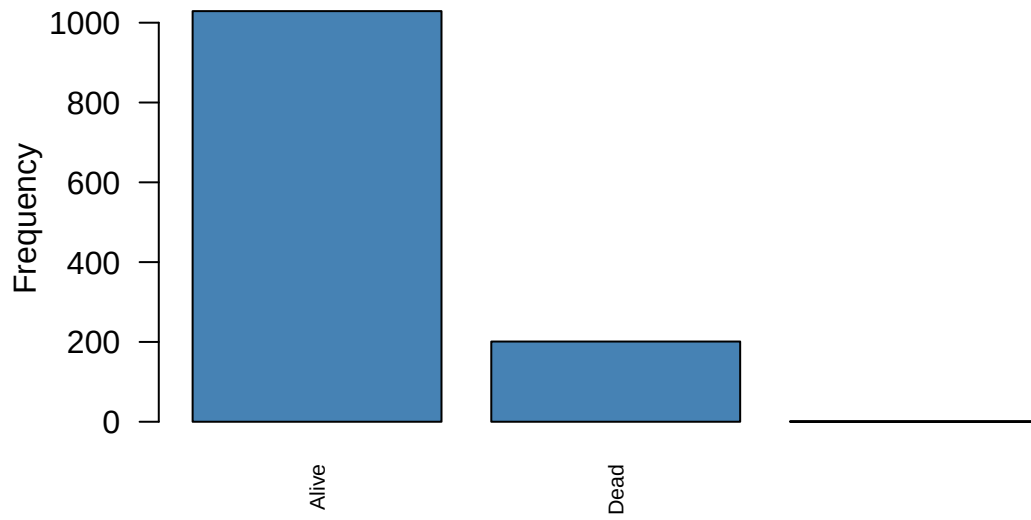
Distribution of gender



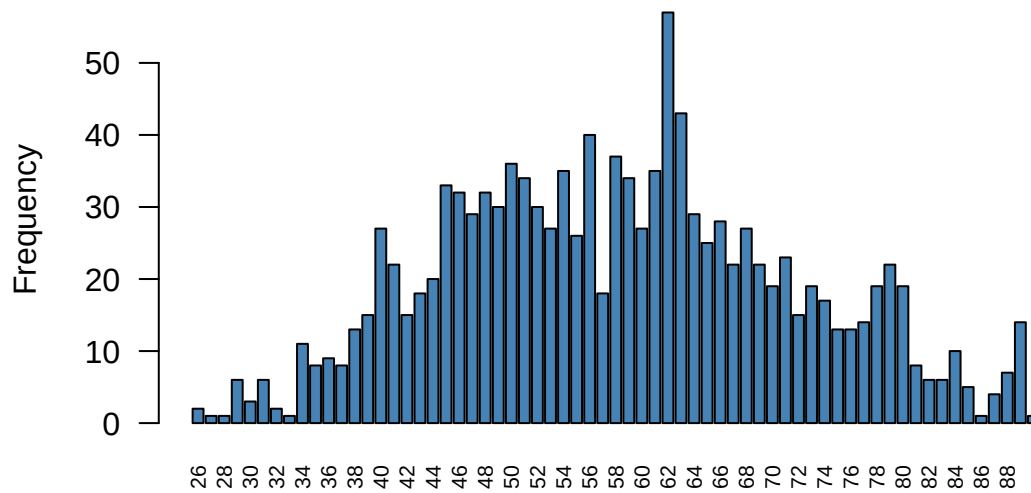
Distribution of ethnicity



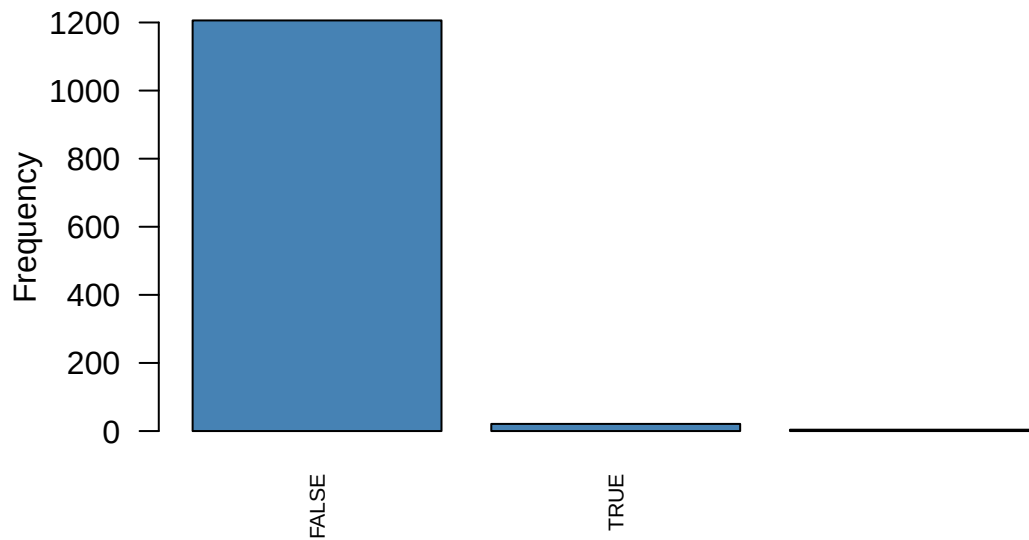
Distribution of vital_status



Distribution of age_at_index



Distribution of age_is_obfuscated



Encoding categorical variables Label encoding of the `clinical_data` categorical variables, with NA also as a category. Label encoding (bijection) for non-integer clinical columns :

```
label_encoded_clinical <- clinical_data
label_mappings <- list()

for (col in names(label_encoded_clinical)) {
  column_values <- label_encoded_clinical[[col]]
  if (!is.numeric(column_values)) {
    if (!is.atomic(column_values)) {
      column_values <- vapply(column_values, function(entry) {
        if (is.null(entry) || (length(entry) == 1 && is.na(entry))) {
          return(NA_character_)
        }
        paste0(as.character(entry), collapse = "|")
      }, character(1))
    }
    factor_values <- factor(column_values, exclude = NULL)
    label_encoded_clinical[[col]] <- as.integer(factor_values)
    label_mappings[[col]] <- levels(factor_values)
  }
}

label_encoded_clinical[] <- lapply(label_encoded_clinical, function(col) {
  if (is.numeric(col)) {
    col[is.na(col)] <- -1 # NA placeholder for numeric columns
  } else {
    f <- factor(col, exclude = NULL)
    col <- as.integer(f)
  }
  col
})

cat("Label-encoded clinical data dimensions:",
```

```

paste(dim(label_encoded_clinical), collapse = " x "), "\n")

## Label-encoded clinical data dimensions: 1231 x 19
if (length(label_mappings) > 0) {
  cat("Bijections:\n")
  for (col in names(label_mappings)) {
    cat(col, "=>",
        paste(seq_along(label_mappings[[col]]),
              label_mappings[[col]],
              sep = ":", collapse = ", "),
        "\n")
  }
}

## Bijections:
## tissue_type => 1:Normal, 2:Tumor
## laterality => 1:Left, 2:Right, 3:NA
## tissue_or_organ_of_origin => 1:Bladder, NOS, 2:Bone marrow, 3:Bone, NOS, 4:Brain, NOS, 5:Breast, NOS
## primary_diagnosis => 1:Adenocarcinoma, NOS, 2:Adenoid cystic carcinoma, 3:Apocrine adenocarcinoma, 4
## prior_treatment => 1:No, 2:Not Reported, 3:Yes, 4:NA
## diagnosis_is_primary_disease => 1:FALSE, 2:TRUE, 3:NA
## ajcc_pathologic_t => 1:T1, 2:T1a, 3:T1b, 4:T1c, 5:T2, 6:T2a, 7:T2b, 8:T3, 9:T3a, 10:T4, 11:T4b, 12:T
## morphology => 1:8010/3, 2:8013/3, 3:8022/3, 4:8050/3, 5:8090/3, 6:8140/3, 7:8200/3, 8:8201/3, 9:8211
## classification_of_tumor => 1:metastasis, 2:not reported, 3:primary, 4:Prior primary, 5:recurrence, 6
## sites_of_involvement => 1:Bone, NOS|Breast, Left Lower Outer|Breast, NOS, 2:Bone, NOS|Breast, Left U
## follow_ups_disease_response => 1:TF-Tumor Free, 2:Unknown, 3:WT-With Tumor, 4:NA
## race => 1:american indian or alaska native, 2:asian, 3:black or african american, 4:not reported, 5:
## gender => 1:female, 2:male, 3:NA
## ethnicity => 1:hispanic or latino, 2:not hispanic or latino, 3:not reported, 4:Unknown, 5:NA
## vital_status => 1:Alive, 2:Dead, 3:NA
## age_is_obfuscated => 1:FALSE, 2:TRUE, 3:NA

```

```
library(glmnet)
```

Logistic RIDGE Regression

```

## Loading required package: Matrix

## Loaded glmnet 4.1-10

predictors <- label_encoded_clinical
predictors$vital_status <- NULL

# Check for NA values in predictors and prints them and Check for NA values in clinical_data$vital_status
if (any(is.na(predictors))) {
  na_rows <- which(rowSums(is.na(predictors)) > 0)
  cat("Rows with NA in predictors:", na_rows, "\n")
}
if (any(is.na(clinical_data$vital_status))) {
  na_rows_vital <- which(is.na(clinical_data$vital_status))
  cat("Rows with NA in vital_status:", na_rows_vital, "\n")
}

## Rows with NA in vital_status: 1010

```

```

# Remove rows with NA in vital_status
model_data <- na.omit(data.frame(predictors, vital_status = clinical_data$vital_status))

# Prepare predictor matrix X (numeric matrix) and response y (factor) for glmnet
X <- as.matrix(model_data[, setdiff(colnames(model_data), "vital_status"), drop = FALSE])
y <- factor(model_data$vital_status)

# Fit a logistic ridge regression model using cross-validation
ridge_fit <- cv.glmnet(X, y, family = "binomial", alpha = 0)
best_lambda <- ridge_fit$lambda.min

# Performs predictions using logistic ridge regression and evaluates the model's performance.
ridge_pred_prob <- predict(ridge_fit, newx = X, s = "lambda.min", type = "response")
threshold <- 0.5
predicted_class <- ifelse(ridge_pred_prob > threshold, levels(y)[2], levels(y)[1])
ridge_conf_mat <- table(Predicted = predicted_class, Observed = y)
ridge_conf_mat

##           Observed
## Predicted Alive Dead
##      Alive   991  115
##      Dead    38   86

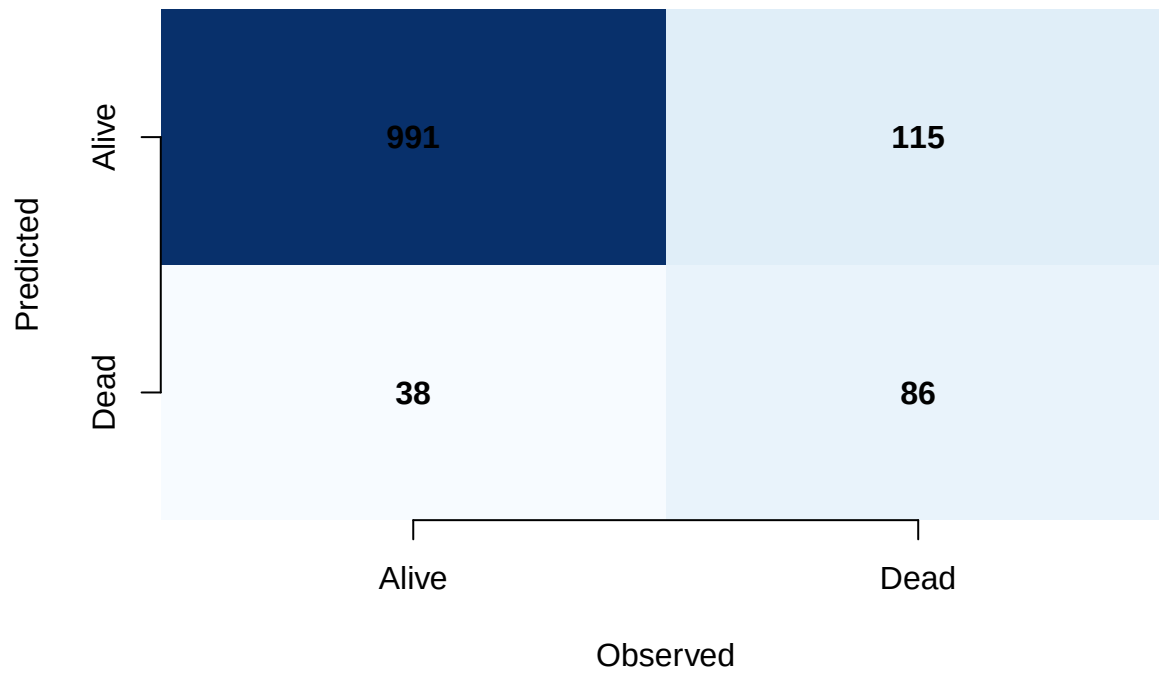
# Compute and print ridge confusion matrix accuracy
ridge_accuracy <- sum(diag(ridge_conf_mat)) / sum(ridge_conf_mat)
cat("Ridge accuracy:", round(ridge_accuracy, 3), "\n")

## Ridge accuracy: 0.876

# Visualization of the ridge confusion matrix
ridge_conf_mat_matrix <- as.matrix(ridge_conf_mat)
image(t(ridge_conf_mat_matrix[nrow(ridge_conf_mat_matrix):1, ]),
      axes = FALSE,
      col = colorRampPalette(c("#f7fbff", "#6baed6", "#08306b"))(64),
      xlab = "Observed",
      ylab = "Predicted",
      main = "Ridge Confusion Matrix")
axis(1,
      at = seq(0, 1, length.out = ncol(ridge_conf_mat_matrix)),
      labels = colnames(ridge_conf_mat_matrix))
axis(2,
      at = seq(0, 1, length.out = nrow(ridge_conf_mat_matrix)),
      labels = rev(rownames(ridge_conf_mat_matrix)))
text(rep(seq(0, 1, length.out = ncol(ridge_conf_mat_matrix)), each = nrow(ridge_conf_mat_matrix)),
      rep(seq(0, 1, length.out = nrow(ridge_conf_mat_matrix)), times = ncol(ridge_conf_mat_matrix)),
      labels = as.vector(ridge_conf_mat_matrix[rev(seq_len(nrow(ridge_conf_mat_matrix))), ]),
      col = "black", font = 2)

```

Ridge Confusion Matrix



Clinical data with Genes

Extension